

Extended Abstract

Motivation Test time scaling methods have become increasingly popular in robot learning, as they allow for flexible guidance in action selection during inference. While increasing compute at test time can have massive benefits, reducing latency is critical, particularly in robot learning, as decisions must be made and executed out in real time in order to feasibly run live on a robot. Because of the high levels of temporal dependence in robot activity, it is promising that we may be able to leverage this to precompute features to reduce latency in future inference steps.

Method In this paper, we experiment with numerous methods to precompute features for more efficient inference. We focus on the bottleneck features of unet-based action diffusion models, as bottleneck features hold rich, compressed, likely more generalizable features. We explore using both precomputed bottleneck features from the predicted next state, as well as simply reusing features from prior timesteps. We explore various methods to determine how prior features should be used, focusing on direct reuse of features depending on a proxy for expected similarity of these features. In our experimentation, we start off with oracle models that have access to the full diffusion processes, then in hindsight determine methods that may have optimally allowed for faster inference. We then move on to prediction of these optimal decisions, and show results on such models.

Implementation In our experiments, we use the PushT environment built upon the BiD diffusion codebase. We use a base unet based action diffusion model and primarily use max rewards per episode as metrics for comparison.

Results We see that the most successful experiments precompute predicted bottleneck features based off of the predicted next state. We use cosine distance to determine the usability of these precomputed features, and periodically recalculate the full diffusion process to calibrate the usability of precomputed features. Our oracle shows a potential of saving up to 60 percent of bottleneck calculations while maintaining similar level of performance as the original, with no precomputation. The final experiments show that high levels performance comparable to maximum compute is possible with up to around 30% use of precomputed bottlenecks.

Discussion The results thus far are incredibly exciting in highlighting the potential of saving compute with even simple methods of feature precomputation. In the future, it would be interesting to explore more rigorous methods of feature prediction, from scaling up features precomputed to more "smartly" using these features other than necessarily reusing them exactly. Furthermore, it is important to eventually apply these techniques to more complex environments, such as more stochastic environments or image-based environments.

Conclusion Overall, this project is exciting in showing the potential of leveraging temporal dependencies in robot policies for reduced latency at inference. While there is plenty of future work to do for this to more reasonably and realistically be deployed, it is exciting to see that even simple methods succeed in reducing latency.

Efficient Prior-Informed Action Diffusion

Swetha Yogeswaran

Department of Computer Science
Stanford University
swethay@stanford.edu

Abstract

Test time scaling methods have become increasingly popular in robot learning, as they allow for flexible guidance in action selection during inference. While increasing compute at test time can have massive benefits, reducing latency is critical, particularly in robot learning, as decisions must be made and executed out in real time in order to feasibly run live on a robot. Because of the high levels of temporal dependence in robot activity, it is promising that we may be able to leverage this to precompute features to allow future inference steps to be more computationally efficient. In this paper, we experiment with methods to precompute features for more efficient inference, focusing on bottleneck features of unet-based action diffusion models. We experiment with oracle policies to determine optimal performance-compute tradeoffs, which show a results comparable to original results with nearly 60% use of precomputed bottlenecks. Finally, we experiment with non-oracle methods which show high results, comparable to the original policy, with around 30% precomputed bottleneck use. Overall, this work is exciting in showing that temporal dependencies can be leveraged to yield high rewards with significantly reduced compute during inference.

1 Introduction

Test time scaling methods have become increasingly popular in robot learning, as they allow for flexible guidance in action selection during inference. For example, algorithms during test time can help a robot maintain temporal consistency, which is important for longer term continuity and planning towards a goal. While increasing compute at test time can have such benefits, reducing latency is critical, particularly in robot learning, as decisions must be made and executed out in real time in order to feasibly run live on a robot.

In order to optimize inference performance with lesser compute, it seems reasonable to leverage temporal patterns during inference time. After all, future states are most definitely dependent on prior states and actions. In standard action diffusion, however, inference at every robot timestep is done independently. Specifically, a robot observes its environment, queries its policy, executes an action, then repeats. Given that there are plenty of intermediate features in a diffusion process, and that environments largely stay the same between diffusion steps and robot timesteps, it seems wasteful to recompute the entire process at every robot and diffusion timestep.

In this work, I explore precomputing features for use during inference time, in order to leverage temporal dependencies to decrease compute at inference time. Specifically, I focus on precomputing the bottleneck of a unet-based action diffusion policy, as these hold a rich, compressed set of features that seem most likely to generalize well across timesteps. I leverage an oracle model to explore upper bounds of performance-compute tradeoffs, and finally dive into a more realistic models that highlight the significant potential of this method.

2 Related Work

2.1 StreamV2V

StreamV2V is a video-to-video diffusion model that maintains temporal consistency through a saved feature bank that compactly encodes information about previous timesteps and informs the generation of future frames Liang et al. (2024). Specifically, the feature bank contains intermediate transformer features that are carefully merged from previous timesteps based on similarity. These features are then used to inform generations of subsequent frames. This has shown massive gains in temporal consistency, highlighting the relevance of prior intermediate features in informing inference at future timesteps. While the focus of this paper is reducing latency at inference as opposed to enforcing temporal consistency, StreamV2V reaffirms the potential of using prior intermediate features in modalities of temporal dependency to inform future inference.

2.2 TeaCache

TeaCache increases diffusion efficiency by caching and reusing model outputs at selective diffusion timesteps, as informed by predicted output similarity. This compression allows for more efficient diffusion without much quality loss. By helping reduce one source of inefficiency, this method opens the door for more complex and beneficial test time scaling approaches that may not have been computationally feasible with inefficient diffusion. While we take inspiration from this in skipping diffusion steps by reusing previous outcomes, the focus of this paper is on leveraging temporal redundancy, so we compress across time instead of compressing across diffusion steps themselves. Liu et al. (2024b)

3 Method

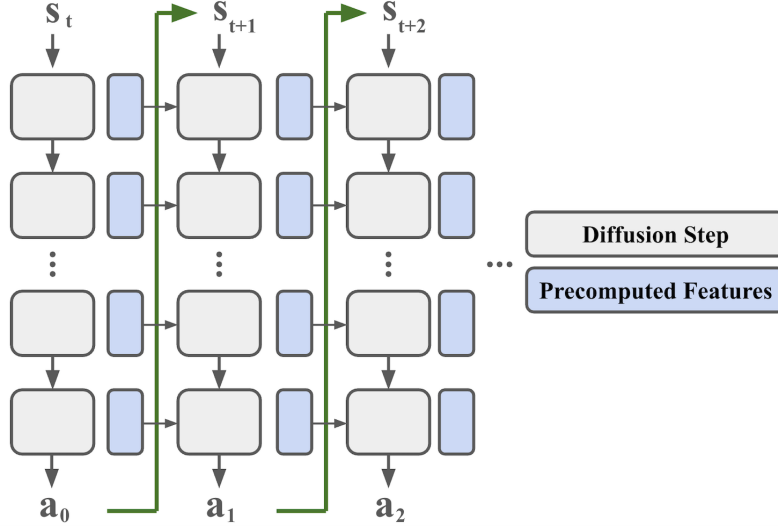


Figure 1: General Framework

In order to leverage the temporal redundancy we expect between robot timesteps, we propose precomputing features or metrics helpful to optimize inference at the next robot timestep. This computation can occur any time before a robot must infer the next action, for example, when a robot is executing its current action. We must then find a smart way to use this precalculated information in the next timestep. For the purpose of this project, we use the following approach: At the first diffusion step, we start in non skip mode. Then, for every following diffusion step, we run the following:

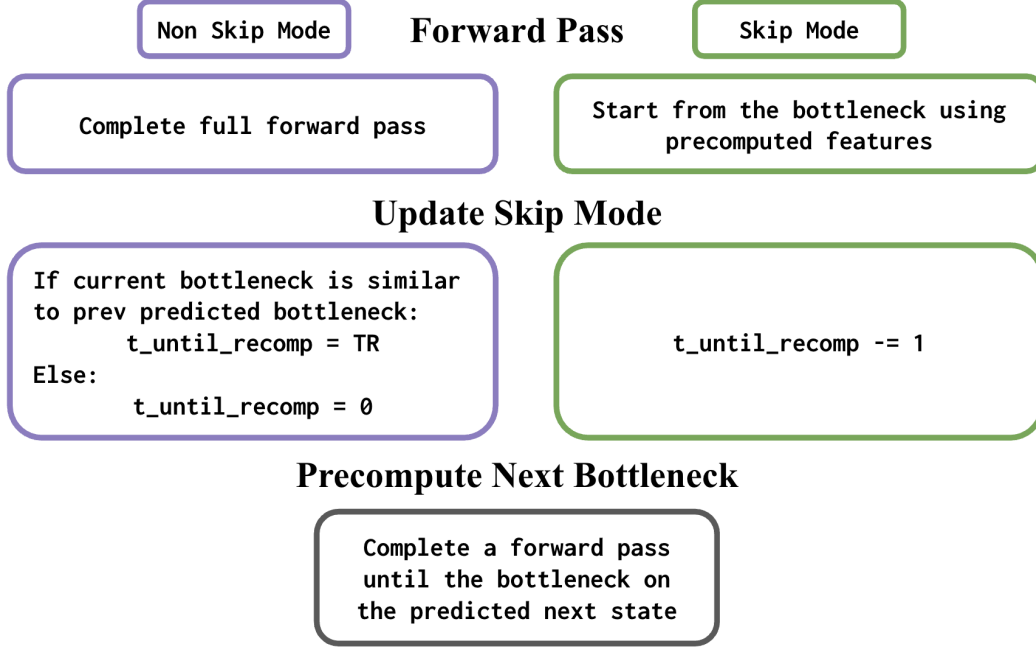


Figure 2: Method at each diffusion step. We switch to skip mode once t until recomp reaches 0. TR is a hyperparameter.

Then, after the diffusion process is complete, we run a forward pass on the predicted next state, saving the bottleneck features for potential reuse at the next robot timestep.

In order to build out this method, I conducted numerous experiments to decide on optimal implementation details, such how to precompute features and use them to inform future timesteps. For clarity of discussion and justification of method components, details of the method are discussed in more depth along with their corresponding experiments in the results section.

4 Experimental Setup

The codebase was built upon the BiD diffusion codebase, specifically using a base unet diffusion model Liu et al. (2024a). Experiments were all run in a state-based PushT simulation environment, where the policy must learn to push a T-shaped object onto an outline. The input into our model is the observation of the state, specifically a 5 dimensional vector $[x_{agent}, y_{agent}, x_{block}, y_{block}, \theta_{block}]$. The output of our model is an action, specifically the 2 dimensional position vector $[x_{agent}, y_{agent}]$ representing the target position of the agent.

To evaluate our method, we focus on maximum reward per episode, where reward is defined as the percent of the T that overlaps with the goal location of the T. This is the standard reward in this environment. We choose maximum reward as opposed to success rate for more finegrained observations of results.

5 Results

5.1 Oracle: Optimal Precomputed Bottlenecks

In this project, I consider two methods of precomputing bottlenecks for the next robot timestep. One, I consider using the bottleneck from the prior timestep itself, motivated by the potential to reuse without any additional computation by purely leveraging temporal redundancy instead of using temporal dependencies to specifically compute a predicted bottleneck. Second, I consider explicitly computing separate predicted bottleneck features through a forward pass on the predicted next state. This state is taken from the output trajectory, as the base diffusion model outputs a trajectory past

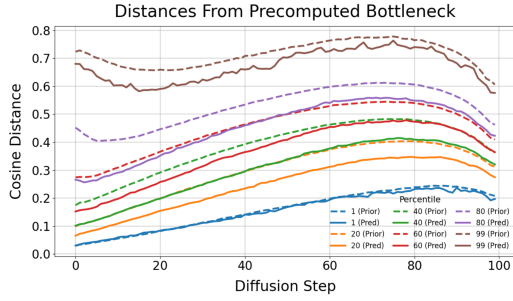


Figure 3: Step-Specific Percentile-Based Cos Distance Thresholds. Dotted shows distance from prior timestep features, while solid shows distance from predicted bottleneck features from the prior timestep.

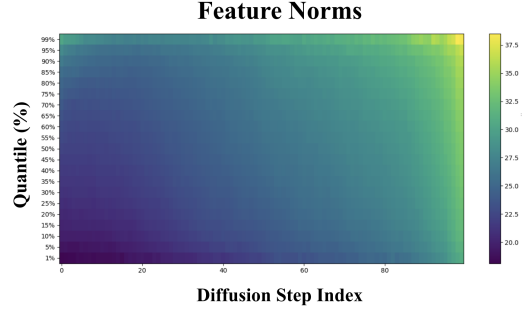


Figure 4: Norms of bottleneck features across diffusion steps. Clearly, these features are not of uniform norm, making L2 difference a metric biased by feature norm.

just the immediate timestep. While this does require additional computation, it can be done before observation of the next state, for example while executing the current action. So, time between observation of a state and deciding on an action can still be reduced through this method. In this oracle experiment, I run the following process per diffusion step:

1. Complete a forward pass until the bottleneck
2. Compare the bottleneck to the bottleneck prior from the previous robot timestep (either predicted bottleneck features based on predicted next state or prior bottleneck itself). If within a threshold cosine distance, reuse the prior bottleneck features
3. Proceed through the forward pass

5.1.1 Threshold Determination

We want to examine the tradeoff between performance and compute efficiency. Specifically, we want to compare the optimal cases of this tradeoff - for example, what happens when we use the top 10%, 50%, or 80% of precomputed features. To determine this threshold, and what constitutes the "top" percentile of features, I explored various thresholding methods.

Distance Metric: I aimed to find a distance metric that served as a proxy for bottleneck feature reusability. To do so, I looked at distance between bottleneck features and precomputed (or prior) corresponding features, hoping to find helpful structures in the data. Specifically, I experimented with L2 distance, normalized L2 difference, and cos distance. While all showed nonrandom structure, making it seem reasonably possible to leverage, I decided against L2 distance, as feature norms themselves have significant variation across diffusion steps, as seen in the figure above, making it hard to determine thresholds that provide clear information about feature similarity in a manner unbiased by feature norms.

Threshold Value: Given that I'll be using cos distance as the distance metric, I now choose thresholds based off of observed cos differences in a single batch. Given that each diffusion step has different distributions of cos distances from prior bottlenecks, I calculate diffusion step-specific thresholds by percentile. While the difference between global and step-specific thresholds can be experimented with, I proceed with step-specific thresholds as initial experimentation showed better results with this method.

5.1.2 Oracle Results: Optimal Precomputed Features

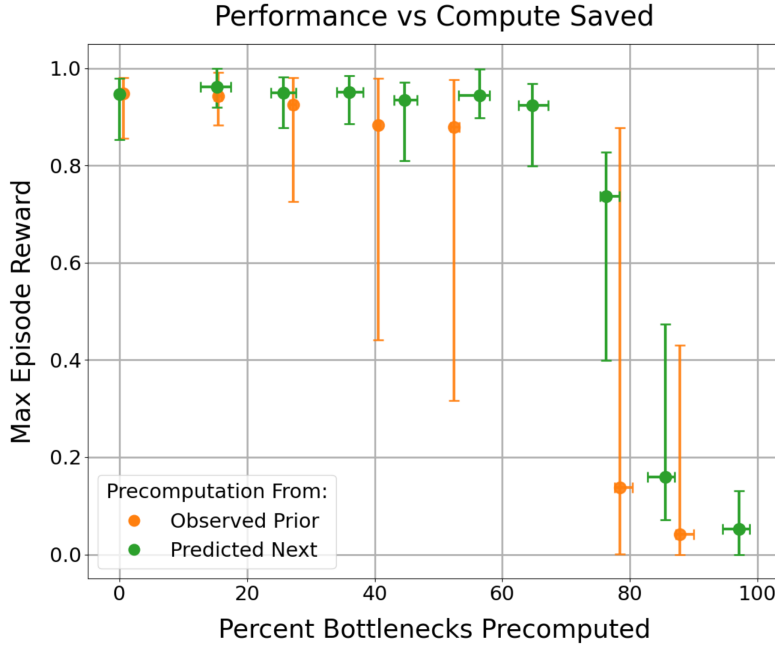


Figure 5: Performance vs Compute Saved of an Oracle Model. This shows results across various thresholds, aggregated by the precomputation method.

In our oracle experiment, we see that using predicted bottleneck features from the predicted next state yields significantly better results than simply reusing bottleneck features from prior states.

It is also interesting to note that with the reusing prior method, the median performance remains high until around 50 percent of the features are reused, although the variance becomes large around 30 percent. This high variance with this method seems reasonable, as the performance in this case is likely associated with states remaining similar to prior states, for example, by slow movement or static behavior. It makes sense that some rollouts might result in more frequent change than others, where reusing prior features may result in compounding lower rewards. Meanwhile, the prediction method of precomputing remains high reward with lower variance until around 60 percent features are reused. It makes sense that this method results in more consistently high rewards for the same amounts of compute saved, since the model factors in change that it expects between states instead of attempting to simply leverage the exact features from before.

5.2 Oracle: Scaling Samples

Given the oracle results from the previous sections, we proceed with precomputing predicted bottlenecks based off of predicted next state, and using diffusion step specific cos distance thresholds. Now, we explore the option of precomputing multiple potential future bottlenecks, with the hope of providing more options for features to optimally reuse. Many options for sampling N potential bottlenecks with some variation were considered:

1. **Run N forward passes using the predicted next state.** This is relatively simple to implement, but has zero variation in predicted next state, which is a source of uncertainty and an area of variance we likely want a more diverse sample from. May not be too much of an issue with 0 stochasticity, though.
2. **Sample N potential next states, and find their corresponding bottlenecks.** Given that this codebase doesn't have a world model built in, it is nontrivial to predict next state separate from our trajectory prediction. Sampling multiple trajectory predictions may be a possibility, but we would likely need to filter reasonable trajectories by closeness of predicted action and action taken.

3. **Introduce Gaussian perturbations of the single bottleneck feature.** While this seems feasible for an oracle model, it seems not too informative for a non-oracle model, as we can likely assume we only can predict bottlenecks within some Gaussian noise. Leveraging closeness by "luck" by actively choosing noisier samples closer to our ground truth model seems unreasonably exploitative for an oracle model.

Given these thoughts, I implemented the first option, as it seemed the most informative in the most straightforward way. I specifically ran experiments with 1, 4, 8, 16, and 32 samples. The results, however, were puzzling and difficult to analyze. For example, the one sample method often reused priors more than the higher sample methods. This oracle experiment is a work in progress, as I'm running some tests and inspecting the code to analyze this further and see if there are errors in the implementation that are causing such odd results.

5.3 Predicting Bottleneck Reusability

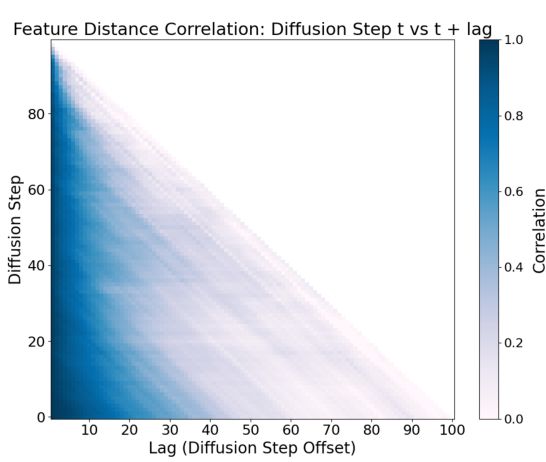


Figure 6: Correlation of Feature Similarity Across Diffusion Steps

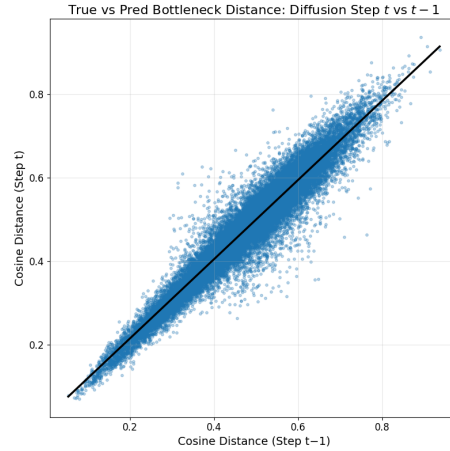


Figure 7: Correlation of feature differences (true vs pred bottleneck) of consecutive diffusion steps

The oracle results show that when features are optimally precomputed and used, we can achieve high levels of performance with lesser compute during inference time. However, in the oracle method, we calculated the bottleneck features at every timestep, and used precomputed features when they were "close enough". This defeats the purpose, as our goal is to not have to compute the bottlenecks in the first place. So, before computing a forward pass, we must predict whether precomputed bottleneck features are close enough to just use directly.

Interestingly, after examining bottleneck distances across diffusion steps, it became clear that distance in bottlenecks in a particular step correlate highly to distance in bottlenecks at closely following diffusion steps. This makes sense, as divergence from prediction at earlier diffusion steps implies that the state or diffusion process diverged in ways that were different than expected, implying we likely should recompute features at nearby diffusion steps as well.

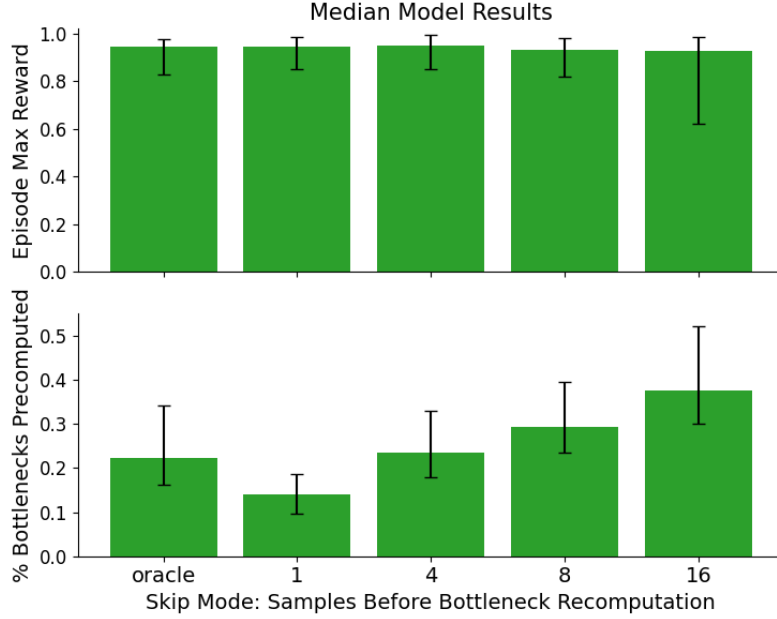


Figure 8: Model Results

These results seem exciting, as we see similar levels of performance with around 30% of the features precomputed. We see performance drop when we do not recompute features for 16 samples after we similar samples which makes sense as previous graphs show low levels of correlation with lags of over 16 diffusion steps. Meanwhile we see dark blue, high correlation, within lags of 10, which shows as skipping up to 8 samples after observing closeness of ground truth and predictions results in similar performance as the oracle while skipping many more steps. In other words, similarity of ground truth vs predicted bottleneck features serves as a reasonable proxy for within 8 diffusion steps.

6 Discussion

Overall, this project is incredibly exciting in revealing the rich temporal dependencies that can be leveraged to maintain high levels of performance with significant computational gain. The following would be interesting to keep in mind as we develop stronger methods to leverage this fundamental idea:

1. **Scaling Precomputation:** Currently, we use a naive approach of simply predicting future bottlenecks using a single predicted future state. More rigorous precomputation methods may precompute more diverse chains of thought that may more efficiently be used in subsequent steps.
2. **Temporal consistency:** While the focus of this work was purely to reduce latency at inference, leveraging temporal dependence to inform future inference can be powerful in enforcing temporal consistency. It may be interesting to consider approaches that are mutually beneficial to the goals of temporal consistency and reduced latency
3. **Stochasticity:** These experiments use highly deterministic environments, which makes prediction of next state much easier than in stochastic environments. It is worthwhile to explore environments where this is not the case, as more realistic environments likely display some level of stochasticity.
4. **Limitations of State-Based Models:** Realistically, robots operate on much more complex observations. While state-based PushT is a nice environment to start on, it is important to move on to more complex image-based models, as their feature spaces are more complex and will likely pose non-trivial challenges in feature alignment and use.

5. **Precomputation Time:** Precomputation must occur before the start of the next robot timestep. We must keep this constraint in mind, especially when diving into more complex, computationally expensive precomputation methods.

7 Conclusion

This project highlights the potential of leveraging temporal dependencies in robot policies for faster inference with similar levels of performance. Specifically, this project walks through simple approaches in reusing and precomputing temporal features in straightforward methods. Even with these simple methods, we see high levels of performance with significant computational gain. With further work to understand what features are most information-dense and temporally predictable, we can devise smarter ways to leverage time-dependence to more rigorously save compute while maintaining high performance.

8 Contributions

This was a individual class project, so I (Swetha Yogeswaran) completed the project independently, implementing and analyzing all experiments and writing the paper. Thanks to the IRIS lab, particular Dr. Yuejiang Liu for incredibly helpful mentorship and discussion throughout this project.

References

- Feng Liang, Akio Kodaira, Chenfeng Xu, Masayoshi Tomizuka, Kurt Keutzer, and Diana Marculescu. 2024. Looking Backward: Streaming Video-to-Video Translation with Feature Banks. *arXiv preprint arXiv:2405.15757* (2024).
- Feng Liu, Shiwei Zhang, Xiaofeng Wang, Yujie Wei, Haonan Qiu, Yuzhong Zhao, Yingya Zhang, Qixiang Ye, and Fang Wan. 2024b. Timestep Embedding Tells: It’s Time to Cache for Video Diffusion Model. *arXiv preprint arXiv:2411.19108* (2024).
- Yuejiang Liu, Jubayer Ibn Hamid, Annie Xie, Yoonho Lee, Maximilian Du, and Chelsea Finn. 2024a. Bidirectional Decoding: Improving Action Chunking via Closed-Loop Resampling. *arXiv preprint arXiv:2408.17355* (2024).